Machine Coding: Multi-Step Form

Problem: Build a multi-step form (About → Education → Experience → Review) with per-step validation, a clickable stepper, Back / Next navigation and a final submit.
Tech stack: React 19 (Vite, JavaScript, Tailwind CSS)
Time limit: 60 minutes

PART A: Thought process

These steps work for any machine coding task:

1. **Clarify** the requirements. Ask before you build; write the assumptions down.
2. **List the interactions** (what the user does → what they see). This is your checklist.
3. **Sketch the components** and who talks to whom.
4. **Design the state**: the smallest set of values; derive everything else.
5. **Define data shapes and contracts** (config, API mock).
6. **Trace one flow** end to end before coding.
7. **Name the hard parts** early so they get time.
8. **Build in layers**: something working on screen first, then add one layer at a time.
9. **Walk through every state** (empty, error, loading, success).
10. **Wrap up**: edge cases, trade-offs, what you'd add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Components + state + config design	7 min
Core build (steps, Next/Back, shared data)	18 min
Validation + stepper locking	12 min
Review page + async submit states	10 min
Accessibility, edge cases, cleanup	5 min
Demo + talk through trade-offs	3 min

Tip: think out loud. The interviewer is grading how you decide, not only the final code. Say "I'm deriving which steps are unlocked instead of storing it, because it can't go stale" — one sentence per decision is enough.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- The form is split into steps: **About** (name, email), **Education** (qualification), **Experience** (years), **Review**.
- Only one step is visible at a time; a stepper shows all steps and the current one.
- **Next** validates the current step and moves forward only if it is valid.
- **Back** moves to the previous step without validating and without losing data.
- Clicking a step in the stepper jumps to it, but you **cannot skip ahead** past an invalid step.
- The **Review** step shows all answers with an **Edit** link per section.
- **Submit** sends the data to an API (mocked), shows loading, then success or an error with retry.
- After success, the user can start again with a clean form.

Non-functional requirements

Area	What we commit to
Performance	Tiny form, no memoisation needed. Validation is O(fields) and runs on render; that's fine.
Scalability	Steps and fields come from **one config array**. Adding a step = adding an object, no new JSX.
Error handling	Field errors next to the input; API error as an alert on Review with a Retry button; no double submit.
Accessibility	Real `<label>`s, `aria-invalid` + `aria-describedby` on errors, `aria-current="step"` in the stepper, focus moves to the step heading on step change and to the first invalid field on failed Next. Enter key works.
Reusability	`Field`, `Stepper`, `validateStep` don't know about this specific form.
Extensibility	New field types (select, textarea), async validators, persistence can be added without changing the flow logic.

Clarifying questions to ask

1. Can the user click any step in the stepper, or only steps they have already completed? *(Affects locking logic.)*
2. Should data survive a page refresh (localStorage / draft API)? *(Persistence is a separate feature.)*
3. When should validation run: on Next only, on blur, or while typing? *(UX + amount of state.)*
4. Is there a Review step before submit, or does the last data step submit directly?
5. What does the submit API look like, and what happens on failure: keep the data and retry?
6. Are the steps fixed, or can they change based on answers (e.g. skip Experience for students)?

Assumptions we build with

- 4 fixed steps; the last one is Review.
- Validate on **Next** (and on Submit); clear a field's error as soon as the user edits it.
- Stepper: you may jump to any step **up to and including the first invalid step**.
- No persistence across refresh (mention as a follow-up).
- Submit is a mocked async call: 800 ms, and an email ending in `@fail.com` simulates a server error.
- On failure the data is kept and the user can retry.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Step 1 "About" shown; only step 1 is clickable in the stepper; Back disabled.
Click **Next** with empty fields	Stay on step; show "Name is required", "Email is required"; focus the Name input.
Type in a field that has an error	That error disappears immediately.
Enter an invalid email and click Next	"Enter a valid email" under Email.
Press **Enter** inside an input on step 1–3	Acts as **Next** (validates), never submits.
Valid step + Next	Move to next step; focus moves to the step heading; stepper updates.
Click **Back**	Previous step; typed values are still there; no validation.
Click a locked step in the stepper	Nothing (button is disabled).
Clear a field on step 1 while later steps were reachable	Later steps become locked again.
On Review, click **Edit** for a section	Jump to that step with its data filled in.
Click **Submit**	Button shows "Submitting…" and is disabled; Back and stepper disabled.
Double-click Submit	Only one request is sent.
API fails	Red alert on Review, button becomes "Retry"; all data kept.
API succeeds	Success card: "Thanks, {name}!" with **Fill another**.
Click **Fill another**	Form resets to step 1 with empty fields.

---

3. Design the architecture

Module	Responsibility
`App`	Page layout, renders the form.
`MultiStepForm`	**Owns all state**: current step, data, errors, submit status. Handles navigation and submit.
`Stepper`	Shows steps, highlights current, calls `onSelect(i)` for unlocked steps. Stateless.
`Field`	One labelled input with its error message and ARIA wiring. Stateless.
`ReviewStep`	Read-only summary grouped by step, with Edit buttons. Stateless.
`config/steps.js`	The steps, their fields and each field's validator. Single source of truth.
`utils/validate.js`	Pure `validateStep(step, data)` and `isStepValid`. Easy to unit test.
`services/submitForm.js`	Mock API (Promise + timeout).

```
App
└── MultiStepForm            (state: step, formData, errors, status)
    ├── Stepper              (props: steps, current, canVisit, onSelect)
    └── <form>
        ├── Field × n        (props: field, value, error, onChange)   ← steps 1-3
        └── ReviewStep       (props: steps, data, onEdit)             ← last step

config/steps.js ──► MultiStepForm, Stepper, Field, ReviewStep, validate.js
services/submitForm.js ──► MultiStepForm (on submit)
```

Start simple: the first version can have hard-coded `AboutStep`, `EducationStep` components. Once you see they're all "label + input + error", move to the config + generic `Field`. Say this out loud: "I'll start with one component per step and refactor to config once the pattern is clear."

4. Design the state / data model

State or field	Purpose
`step` (number)	Index of the visible step.
`formData` (object)	All answers for **all** steps, keyed by field name. Lives in the parent so it survives step changes.
`errors` (object)	Error messages for the **current** step's fields, shown after Next/Submit.
`status` (`'idle' | 'submitting' | 'success' | 'error'`)	Submit lifecycle; one string instead of three booleans.
`submitError` (string)	Message from the API failure.

```
const [step, setStep] = useState(0);
const [formData, setFormData] = useState(INITIAL_DATA); // { name:'', email:'', education:'',
experience:'' }
const [errors, setErrors] = useState({});
const [status, setStatus] = useState('idle');          // idle | submitting | success | error
const [submitError, setSubmitError] = useState('');
```

Senior note: derive, don't store.

- `firstInvalidStep = STEPS.findIndex(s => !isStepValid(s, formData))` is **derived** from `formData`. You don't need a `completedSteps` or `maxStepReached` state; that state would go stale when the user goes back and clears a field.
- `canVisit(i)`, `isLast`, `submitting`, the button label and the progress text are all derived.
- `INITIAL_DATA` is derived from the config, so adding a field can't forget the initial value.
- **Ownership:** `MultiStepForm` owns everything. Steps are unmounted when hidden, so any state inside them would be lost; that's why `formData` is lifted up.

---

5. Design the data structures and contracts

Step config (stable `id`s for React keys, `name`s for data keys)

```
export const STEPS = [
  { id: 'about', title: 'About', fields: [
    { name: 'name',  label: 'Full name', type: 'text',  autoComplete: 'name',  validate: (v) => ... },
    { name: 'email', label: 'Email',     type: 'email', autoComplete: 'email', validate: (v) => ... },
  ]},
  { id: 'education',  title: 'Education',  fields: [{ name: 'education', ... }] },
  { id: 'experience', title: 'Experience', fields: [{ name: 'experience', type: 'number', min: 0, max:
50, ... }] },
```

```
  { id: 'review',     title: 'Review',     fields: [] },
];
```

Why: one array drives the stepper labels, which inputs render, validation and the review summary. Validators return an **error string or** `''`, so `validateStep` is a simple loop.

Contracts

Function	Input	Output
`field.validate(value)`	string	`''` if valid, else the message
`validateStep(step, data)`	step config, form data	`{ email: 'Enter a valid email' }` (only invalid fields)
`isStepValid(step, data)`	same	`true / false`
`submitForm(data)`	`{ name, email, education, experience }`	`Promise<{ id, ...data }>` ; rejects with `Error('Server error, please try again.')`

Example request / response of the mock:

```
// request
{ "name": "Asha", "email": "asha@x.com", "education": "B.Tech", "experience": "3" }
// 200
{ "id": "7c1e…", "name": "Asha", "email": "asha@x.com", "education": "B.Tech", "experience": "3" }
// failure (email ends with @fail.com)
Error: "Server error, please try again."
```

Note: `experience` stays a **string** in state because inputs give strings; convert with `Number()` only when validating or sending.

---

6. Trace the data flow

Interaction: user is on step 1, fills Name only, clicks Next, fixes Email, clicks Next.

```
Click "Next" (type="submit")
        │
        ▼
form onSubmit → handleSubmit(e) → e.preventDefault()
        │  step < LAST → goNext()
        ▼
validateStep(STEPS[0], formData)  →  { email: 'Email is required' }
        │
```

```
        ▼
setErrors({ email: ... })  +  document.getElementById('email').focus()
        |
        ▼
Render: Field(email) gets error → red border, message, aria-invalid=true
        |
User types "asha@x.com"
        ▼
updateField → setFormData({...prev, email}) + clear errors.email
        |  re-render: firstInvalidStep becomes 1 → stepper unlocks step 2
        ▼
Click "Next" → validateStep → {} → setStep(1)
        |
        ▼
useEffect([step]) → focus <h2> "Step 2 of 4: Education"
```

---

7. Important technical challenges

Challenge 1: Data must survive step changes

Problem: If each step keeps its own `useState`, going Back unmounts the current step and its values are gone. Scenario: fill Education, click Back, click Next → Education is empty.

Solution: Lift all values into one `formData` object in the parent, keyed by field `name`. One generic `onChange` updates any field.

```
const updateField = (e) => {
  const { name, value } = e.target;
  setFormData((prev) => ({ ...prev, [name]: value }));
  setErrors((prev) => (prev[name] ? { ...prev, [name]: '' } : prev)); // clear stale error
};
```

Challenge 2: Stepper must not let users skip validation

Problem: A common first version renders tabs like `onClick={() => setStep(2)}`. Scenario: on step 1 with empty fields, click "Experience", then submit → invalid data reaches the API.

Solution: Derive which steps are reachable from the data itself. You can visit any step up to the first invalid one. No extra state, and it re-locks when a field is cleared. Re-validate everything on submit as a last guard.

```
const firstInvalidStep = STEPS.findIndex((s) => !isStepValid(s, formData));
const canVisit = (i) => !submitting && (firstInvalidStep === -1 || i <= firstInvalidStep);

// in handleSubmit, on the last step:
```

```
if (firstInvalidStep !== -1) {
  setStep(firstInvalidStep);
  setErrors(validateStep(STEPS[firstInvalidStep], formData));
  return;
}
```

Challenge 3: Enter key and double submit

Problem: Inside a `<form>`, pressing Enter in any input triggers `onSubmit`. Scenario: user presses Enter on step 1 → the form "submits" half-empty data. Also, attaching both `onClick={handleSubmit}` on the button **and** `onSubmit={handleSubmit}` on the form runs the handler twice per click; double-clicking during the request sends two requests.

Solution: One `onSubmit` handler that decides by step: middle step → Next; last step → submit. Only `onSubmit`, no `onClick`. A `status` guard plus a disabled button stop duplicates.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  if (step < LAST) return goNext();   // Enter = Next on middle steps
  if (submitting) return;             // double-click guard
  setStatus('submitting');
  try { await submitForm(formData); setStatus('success'); }
  catch (err) { setStatus('error'); setSubmitError(err.message); }
};
```

Challenge 4: Focus and screen-reader feedback on step change

Problem: When the step changes, the old inputs disappear and focus falls back to `<body>`. A keyboard or screen-reader user has no idea a new step appeared, or which field failed.

Solution: Focus the step heading (`tabIndex={-1}`) whenever `step` changes; on failed Next, focus the first invalid input. Link errors with `aria-describedby`.

```
useEffect(() => { headingRef.current?.focus(); }, [step]);

const firstField = Object.keys(stepErrors)[0];
if (firstField) document.getElementById(firstField)?.focus();

<input aria-invalid={Boolean(error)} aria-describedby={error ? `${name}-error` : undefined} />
```

---

8. Implement in the correct order

#	Step	You can demo
1	`MultiStepForm` with `step` state and three hard-coded step blocks, Back/Next buttons	Clicking through steps
2	Lift `formData` into the parent, one `updateField`	Values survive Back/Next
3	Move steps/fields into `config/steps.js`, render with a generic `Field`	Same UI, less JSX
4	Add `validate` per field + `validateStep`; Next blocked when invalid	Error messages
5	Clear a field's error on change; focus first invalid field	Smooth error UX
6	`Stepper` with `aria-current` and derived `canVisit`	Locked / unlocked steps
7	Single `onSubmit` handler: Enter = Next, last step = submit	Keyboard works
8	`ReviewStep` with Edit buttons	Summary page
9	Mock `submitForm`, `status` state, disabled buttons, error + Retry	Loading / error / success
10	Success card + reset	Full happy path
11	Heading focus on step change, ARIA on fields	Accessible
12	Edge-case pass (whitespace, decimals, double click)	Robust

9. Cover all states and edge cases

State / case	What the user sees	How
Initial	Step 1, empty fields, only step 1 clickable, Back disabled	`step=0`, derived `canVisit`
Field error	Red border + message, focus on first bad field	`errors` + `aria-invalid`
Fixing a field	Error disappears on first keystroke	`updateField` clears `errors[name]`
Whitespace-only input `" "`	Treated as empty	`.trim()` in validators
Name with 1 letter	"Name is too short"	length check
Email `a@b`	"Enter a valid email"	regex
Experience `2.5`, `-1`, `51`	"Enter whole years between 0 and 50"	`Number.isInteger`, range
Experience `e` typed in number input	Browser reports `''` → "Experience is required"	number inputs return `''` for invalid text
Back from any step	Values kept, no errors shown	data in parent, `goTo` clears errors
Earlier field cleared	Later steps lock again	derived `firstInvalidStep`
Enter key on step 1–3	Same as Next	`handleSubmit` branches by step
Submitting	"Submitting…", buttons + stepper disabled	`status='submitting'`
Double click Submit	One request	status guard + disabled
API error	Alert on Review, "Retry" button, data kept	`status='error'`, `submitError`
Success	"Thanks, Asha!" + Fill another	`status='success'`
Reset	Back to clean step 1	`reset()`
Narrow screen	Card is `w-full max-w-lg`, no fixed width	Tailwind classes

10. Final structure

```
_23_multi_step_form/
├── index.html
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css              @import "tailwindcss";
    ├── App.jsx
    ├── config/
    │   └── steps.js           steps, fields, validators, INITIAL_DATA
```

```
    ├── utils/
    │   └── validate.js           validateStep, isStepValid (pure)
    ├── services/
    │   └── submitForm.js         mock API
    └── components/
        ├── MultiStepForm.jsx     state + navigation + submit
        ├── Stepper.jsx
        ├── Field.jsx
        └── ReviewStep.jsx
```

---

11. Full working code

`src/config/steps.js`

```
const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

// One source of truth: the stepper, the inputs, validation and the review page all read this.
export const STEPS = [
  {
    id: 'about',
    title: 'About',
    fields: [
      {
        name: 'name', label: 'Full name', type: 'text', autoComplete: 'name',
        validate: (v) => (!v.trim() ? 'Name is required' : v.trim().length < 2 ? 'Name is too short' :
''),
      },
      {
        name: 'email', label: 'Email', type: 'email', autoComplete: 'email',
        validate: (v) => (!v.trim() ? 'Email is required' : !EMAIL_RE.test(v.trim()) ? 'Enter a valid
email' : ''),
      },
    ],
  },
  {
    id: 'education',
    title: 'Education',
    fields: [
      {
        name: 'education', label: 'Highest qualification', type: 'text',
        validate: (v) => (!v.trim() ? 'Qualification is required' : ''),
      },
    ],
  },
  {
```

```
    id: 'experience',
    title: 'Experience',
    fields: [
      {
        name: 'experience', label: 'Years of experience', type: 'number', min: 0, max: 50,
        validate: (v) => {
          if (v === '') return 'Experience is required';
          const n = Number(v);
          return Number.isInteger(n) && n >= 0 && n <= 50 ? '' : 'Enter whole years between 0 and 50';
        },
      },
    ],
  },
  { id: 'review', title: 'Review', fields: [] },
];

export const INITIAL_DATA = Object.fromEntries(
  STEPS.flatMap((s) => s.fields).map((f) => [f.name, ''])
);
```

src/utils/validate.js

```
// Pure: (step, data) -> { fieldName: message } for invalid fields only.
export function validateStep(step, data) {
  const errors = {};
  for (const field of step.fields) {
    const message = field.validate?.(data[field.name]);
    if (message) errors[field.name] = message;
  }
  return errors;
}

export const isStepValid = (step, data) => Object.keys(validateStep(step, data)).length === 0;
```

src/services/submitForm.js

```
// Mock API: resolves after 800ms; an email ending in @fail.com simulates a server error.
export function submitForm(data) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (data.email.trim().endsWith('@fail.com')) reject(new Error('Server error, please try again.'));
      else resolve({ id: crypto.randomUUID(), ...data });
    }, 800);
  });
}
```

`src/components/MultiStepForm.jsx`

```
import { useEffect, useRef, useState } from 'react';
import { STEPS, INITIAL_DATA } from '../config/steps.js';
import { validateStep, isStepValid } from '../utils/validate.js';
import { submitForm } from '../services/submitForm.js';
import Stepper from './Stepper.jsx';
import Field from './Field.jsx';
import ReviewStep from './ReviewStep.jsx';

const LAST = STEPS.length - 1;

const MultiStepForm = () => {
  const [step, setStep] = useState(0);
  const [formData, setFormData] = useState(INITIAL_DATA);
  const [errors, setErrors] = useState({});
  const [status, setStatus] = useState('idle'); // idle | submitting | success | error
  const [submitError, setSubmitError] = useState('');
  const headingRef = useRef(null);

  // Derived, not stored: steps after the first invalid one are locked.
  const firstInvalidStep = STEPS.findIndex((s) => !isStepValid(s, formData));
  const submitting = status === 'submitting';
  const canVisit = (i) => !submitting && (firstInvalidStep === -1 || i <= firstInvalidStep);

  // Announce the new step to screen readers and keyboard users.
  useEffect(() => {
    headingRef.current?.focus();
  }, [step]);

  const updateField = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
    setErrors((prev) => (prev[name] ? { ...prev, [name]: '' } : prev));
  };

  const goTo = (i) => {
    setErrors({});
    setStep(i);
  };

  const goNext = () => {
    const stepErrors = validateStep(STEPS[step], formData);
    setErrors(stepErrors);
    const firstField = Object.keys(stepErrors)[0];
    if (firstField) document.getElementById(firstField)?.focus();
    else setStep(step + 1);
  };
```

```
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (step < LAST) return goNext(); // Enter on a middle step means "Next", not "Submit"
    if (submitting) return;
    if (firstInvalidStep !== -1) {
      setStep(firstInvalidStep);
      setErrors(validateStep(STEPS[firstInvalidStep], formData));
      return;
    }
    setStatus('submitting');
    setSubmitError('');
    try {
      await submitForm(formData);
      setStatus('success');
    } catch (err) {
      setStatus('error');
      setSubmitError(err.message);
    }
  };

  const reset = () => {
    setFormData(INITIAL_DATA);
    setErrors({});
    setStatus('idle');
    setStep(0);
  };

  if (status === 'success') {
    return (
      <div className='w-full max-w-lg rounded-lg bg-white p-6 text-center shadow-xl' role='status'>
        <h1 className='text-xl font-bold'>Thanks, {formData.name.trim()}!</h1>
        <p className='mt-2 text-gray-600'>Your details were submitted.</p>
        <button onClick={reset} className='mt-6 cursor-pointer rounded bg-blue-600 px-6 py-2 text-
white'>
          Fill another
        </button>
      </div>
    );
  }

  const current = STEPS[step];

  return (
    <div className='w-full max-w-lg rounded-lg bg-white p-6 shadow-xl'>
      <h1 className='text-center text-xl font-bold'>Multi-step form</h1>
      <Stepper steps={STEPS} current={step} canVisit={canVisit} onSelect={goTo} />

      <form onSubmit={handleSubmit} noValidate className='mt-6'>
```

```
        <h2 ref={headingRef} tabIndex={-1} className='mb-4 text-lg font-semibold outline-none'>
          Step {step + 1} of {STEPS.length}: {current.title}
        </h2>

        {step === LAST ? (
          <ReviewStep steps={STEPS} data={formData} onEdit={goTo} />
        ) : (
          current.fields.map((f) => (
            <Field key={f.name} field={f} value={formData[f.name]} error={errors[f.name]} onChange=
{updateField} />
          ))
        )}

        {step === LAST && status === 'error' && <p role='alert' className='mt-4 text-sm text-red-600'>
{submitError}</p>}

        <div className='mt-6 flex justify-between'>
          <button
            type='button'
            onClick={() => goTo(step - 1)}
            disabled={step === 0 || submitting}
            className='cursor-pointer rounded bg-gray-200 px-6 py-2 disabled:cursor-not-allowed
disabled:opacity-50'
          >
            Back
          </button>
          <button
            type='submit'
            disabled={submitting}
            className='cursor-pointer rounded bg-blue-600 px-6 py-2 text-white disabled:opacity-60'
          >
            {step < LAST ? 'Next' : submitting ? 'Submitting…' : status === 'error' ? 'Retry' :
'Submit'}
          </button>
        </div>
      </form>
    </div>
  );
};

export default MultiStepForm;
```

`src/components/Stepper.jsx`

```
const Stepper = ({ steps, current, canVisit, onSelect }) => (
  <nav aria-label='Form progress' className='mt-4'>
    <ol className='flex gap-2'>
```

```
      {steps.map((step, i) => (
        <li key={step.id} className='flex-1'>
          <button
            type='button'
            onClick={() => onSelect(i)}
            disabled={!canVisit(i)}
            aria-current={i === current ? 'step' : undefined}
            className={`w-full rounded py-2 text-sm ${
              i === current ? 'bg-blue-600 text-white' : i < current ? 'bg-blue-100' : 'bg-gray-100'
            } cursor-pointer disabled:cursor-not-allowed disabled:opacity-50`}
          >
            {i + 1}. {step.title}
          </button>
        </li>
      ))}
    </ol>
  </nav>
);

export default Stepper;
```

`src/components/Field.jsx`

```
const Field = ({ field, value, error, onChange }) => {
  // eslint-disable-next-line no-unused-vars -- label/validate are config, not input attributes
  const { label, validate, ...inputProps } = field;
  const errorId = `${field.name}-error`;

  return (
    <div className='mb-4'>
      <label htmlFor={field.name} className='mb-1 block font-medium'>{label}</label>
      <input
        {...inputProps}
        id={field.name}
        value={value}
        onChange={onChange}
        aria-invalid={Boolean(error)}
        aria-describedby={error ? errorId : undefined}
        className={`w-full rounded border p-2 ${error ? 'border-red-500' : 'border-gray-300'}`}
      />
      {error && <p id={errorId} className='mt-1 text-sm text-red-600'>{error}</p>}
    </div>
  );
};

export default Field;
```

`src/components/ReviewStep.jsx`

```
import { Fragment } from 'react';

const ReviewStep = ({ steps, data, onEdit }) => (
  <div className='space-y-3'>
    {steps.map((step, i) => step.fields.length > 0 && (
      <section key={step.id} className='rounded border border-gray-200 p-3'>
        <div className='mb-2 flex items-center justify-between'>
          <h3 className='font-semibold'>{step.title}</h3>
          <button type='button' onClick={() => onEdit(i)} className='cursor-pointer text-sm text-blue-
600 underline'>
            Edit
          </button>
        </div>
        <dl className='grid grid-cols-2 gap-1 text-sm'>
          {step.fields.map((f) => (
            <Fragment key={f.name}>
              <dt className='text-gray-500'>{f.label}</dt>
              <dd className='break-words'>{data[f.name]}</dd>
            </Fragment>
          ))}
        </dl>
      </section>
    ))}
  </div>
);

export default ReviewStep;
```

`src/App.jsx`

```
import MultiStepForm from './components/MultiStepForm.jsx';

const App = () => (
  <main className='flex min-h-screen items-center justify-center bg-gray-900 p-4'>
    <MultiStepForm />
  </main>
);

export default App;
```

`main.jsx` and `index.css` (`@import "tailwindcss";`) stay as created by Vite.

---

12. Wrap-up

Edge cases handled

- Data survives Back / Next / stepper jumps (state lifted to the parent).
- Stepper cannot skip an invalid step; it re-locks if an earlier field is cleared.
- Enter key behaves as Next on middle steps; Submit runs once (no `onClick` + `onSubmit` double call, status guard, disabled button).
- Whitespace-only values, invalid email, decimal / out-of-range experience.
- API failure keeps data and offers Retry; success shows a confirmation and a reset.
- Focus moves to the new step heading and to the first invalid field; errors are linked with ARIA.

Trade-offs to mention

- **Config-driven vs one component per step.** Config scales and removes repeated JSX, but a step with a very custom layout needs an escape hatch (e.g. `component` in the config).
- **Validate on Next vs on blur/typing.** Less noise for the user and less state (`touched` not needed); the cost is later feedback.
- **Derived `canVisit`** runs every validator on each render. Trivial here; for big forms memoise with `useMemo`.
- **`useState` vs `useReducer`.** Five pieces of state are fine with `useState`; with more transitions (async validation, autosave) a reducer makes them explicit.
- **No form library.** React Hook Form + Zod would be the production choice; here it shows understanding and avoids a dependency.
- `document.getElementById` for focus is simple; refs per field would be the "pure React" way but need a ref map.

What to add with more time

- **Persistence:** save `formData` + `step` to `localStorage` (debounced) and restore on load; clear on success.
- **Tests:** unit tests for `validateStep`; React Testing Library for "Next blocked when invalid", "Enter = Next", "double click sends once", "stepper locked".
- **Async field validation** (e.g. "email already registered") with debounce and request cancellation.
- **Conditional steps** (skip Experience for students) by adding `when: (data) => boolean` to the config.
- **Progress bar**, unsaved-changes warning (`beforeunload`), and URL sync (`?step=2`) so Back button of the browser works.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you ask whether users can jump to any step?

It decides the locking logic. "Free navigation" means validating everything at submit; "only completed steps" needs the `firstInvalidStep` rule. Building the wrong one costs time to undo.

Q2. What if the steps depend on earlier answers (e.g. students skip Experience)?

Add `when: (data) => boolean` to each step and compute `visibleSteps = STEPS.filter(s => !s.when || s.when(formData))`. Use `visibleSteps` everywhere instead of `STEPS`. Keep the data of hidden steps or drop it on submit, depending on the product.

State design

Q3. Why not keep each step's values in its own component state?

Hidden steps are unmounted, so their state is destroyed. The parent owns the data; children are controlled inputs.

Q4. Why not store `completedSteps` or `maxStepReached`?

It can be derived from `formData`, and stored copies go stale: if the user clears Name on step 1, a stored "step 1 completed" is now wrong. Derived values are always correct.

Q5. Why a `status` string instead of `isLoading`, `isError`, `isSuccess`?

Three booleans allow impossible combos (`isLoading && isSuccess`). One string has exactly the four valid states.

Q6. Would you use `useReducer` here?

At this size `useState` is clearer. I'd switch when transitions grow (async validation, autosave, undo), because a reducer puts all transitions in one testable function: `dispatch({ type: 'NEXT' })`.

Challenge: shared data and validation

Q7. Why are validators in the config and not in the component?

The rule belongs to the field. `validateStep` stays a generic pure loop, is easy to test with plain objects, and the same rules can run on the server (or be replaced by a Zod schema).

Q8. Why clear the error on change but only validate on Next?

Showing "required" while someone is still typing is noisy. Clearing on change gives immediate positive feedback. A common middle ground is validate on blur after the first failed submit.

Q9. The experience input is `type="number"`. Why is the value still a string?

DOM inputs always return strings, and invalid text in a number input returns `''`. Keeping strings avoids `NaN` in state and keeps the input controlled; convert with `Number()` in the validator or before sending.

Challenge: stepper and submit

Q10. What stops someone from skipping validation via the stepper?

Steps after the first invalid one are disabled (`canVisit`), and `handleSubmit` re-checks all steps and jumps to the first invalid one. The UI lock is UX; the submit check is the guard (and the server validates again).

Q11. Why does pressing Enter on step 1 not submit the form?

`onSubmit` branches: `if (step < LAST) return goNext()`. Enter = Next is what users expect, and it reuses the same validation.

Q12. How do you prevent double submission?

`if (submitting) return` in the handler plus `disabled` on the button. Also, only `onSubmit` on the form, not `onClick` on the submit button as well, which would call the handler twice.

Q13. What if the API is slow or the user leaves the page mid-request?

Show the loading state (already done) and add a timeout with `AbortController`. If the component unmounts, abort the request so `setStatus` isn't called on an unmounted component. For really slow APIs, consider optimistic success with a background retry only if the business allows.

Q14. The submit failed. What happens to the data?

Nothing is lost: `formData` is untouched, the user sees the error on Review and the button becomes Retry. We only reset after success.

Implementation steps

Q15. Why do you focus the heading on step change?

When the old inputs unmount, focus drops to `<body>`. Focusing a `tabIndex={-1}` heading tells screen readers "Step 2 of 4: Education" and puts keyboard users at the top of the new step.

Q16. How would you add a `select` or `textarea` field?

Add `as: 'select'` and `options` to the field config and branch in `Field` on `field.as`. The rest (state, validation, review) already works by `name`.

Q17. How would you persist progress across a refresh?

Lazy-init state from storage: `useState(() => JSON.parse(localStorage.getItem(KEY)) ?? INITIAL_DATA)` and save in a debounced `useEffect`. Never store sensitive fields (passwords, card numbers). Clear the key on success.

Performance and scale

Q18. What if the form had 20 steps and 200 fields?

Typing re-renders only the visible step's fields (10 or so), so that's fine. The derived `firstInvalidStep` runs all validators each render; memoise it with `useMemo([formData])` or validate only on Next and store `validSteps`. For very large forms, a library like React Hook Form uses uncontrolled inputs to avoid re-renders per keystroke.

Q19. Every keystroke re-renders the whole form. Is that a problem?

Not at this size: four small components. If profiling showed it, wrap `Field` in `React.memo` and keep `updateField` stable with `useCallback` (it already uses the functional `setState`, so it has no dependencies).

Trade-offs / "why not X?"

Q20. Why not use React Hook Form or Formik?

In production I would (plus Zod for schemas). In an interview, building it shows I understand controlled inputs, validation flow and focus management, and it avoids setup time and a dependency.

Q21. Why not render all steps and hide them with CSS?

It keeps values in the DOM, but hidden inputs still take part in Tab order and native validation unless handled carefully, and it renders everything up front. With state in the parent, unmounting is simpler and cheaper.

Q22. Why not put the step in the URL?

Good idea with more time: `?step=2` makes the browser Back button and refresh work. It needs guarding so a user can't load `?step=4` with empty data; the same `canVisit` rule handles that.